AIDA vs Claude Code Workflows (`/workflows`)

AIDA positioning · best-effort comparison — live source: github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09 — Claude Code's Workflows surface (the JS-orchestration `Workflow` tool, `/workflows` viewer, saved-workflow slash commands, the `agent()`/`parallel()`/`pipelin *hooks, the 1000-agent cap) is young and moving fast. Re-verify against the current Claude Code docs before treating capability claims here as authoritative; if the surface drifts in a way that changes the layer story, update this doc. Companion to vs-claude-code-subagents.md.*

TL;DR: A Claude Code **Workflow** is a *within-task* orchestration mechanism — a JavaScript script that fans out dozens-to-hundreds of subagents, holds the plan *in code* (not in Claude's context), records intermediate results in script variables, and ends with a single answer or artifact. **AIDA** is a *cross-session* substrate + lifecycle — a persistent git-canonical requirement graph with stable IDs, code→spec traces, and an autonomous drain that ships real PRs and tracks spec state over time, across agents and vendors. **Different units of work** (a *task's answer* vs a *spec's lifecycle*). They overlap on the word "orchestration," they **compose** in two directions, and the part that overlaps — the orchestration *mechanism* — is commoditizing in AIDA's favor, not against it.

The worry that prompts this doc: both ship something called an "orchestrator," both fan out parallel agents, both run in the background, both talk about "phases" and "pipelines." A reasonable reader asks *am I getting the same thing twice?* The honest answer is **no** — but the rhyme is loud, so naming the boundary out loud is what makes it stick.

What each one actually is

	Claude Code Workflows (`/workflows`)	AIDA (graph + drain)
Unit of work	One **task**, decomposed across many agents	One **spec**, moved through its development lifecycle
What it produces	An answer / artifact (a report, a reviewed diff, a migration), then the run ends	A shipped PR + a spec that flips `Draft → … → Completed`, persisted in git

	Claude Code Workflows (`/workflows`)	AIDA (graph + drain)
Where the plan lives	In the **script** (fixed at write time); the runtime holds the loop	In the **phase machine + queue + lease state** on disk
Lifetime	**Task-ephemeral** — run once, results in script variables, ends. Close the session and the run is gone	**Cross-session** — the spec, queue entry, lease, status, and trace comments survive every session ending
Persistent state	None across sessions (resume is within one session only)	Git-canonical: orphan `aida-store` branch, YAML per spec, history, traces in source
Identity	None — agents know the prompt, not your requirement graph	Stable **SPEC-IDs** that survive renames, merges, and vendor switches
Git-aware	The *script* isn't; spawned subagents can edit/push, but there's no branch/PR/merge/lifecycle model	Branches, PRs, auto-bump-on-merge, worktree-isolated implementers — the whole lifecycle
Escalation	None — the script logic decides, or the run pauses for a *permission* prompt (not a judgment call)	implementer → advisor → human ladder; punt handshake; reviewer verdicts
Vendor	Claude Code only	Cross-vendor — Claude, Codex, Antigravity all drive one git-canonical substrate
Scale	Dozens-to-100s of agents in one run (1000 cap, ~16 concurrent)	One full lifecycle per spec; parallelism across worktrees via leases
The shape	**A scriptable fan-out you invoke**	**A durable substrate + a lifecycle that runs on it**

A Workflow is a *callable that scales out*. AIDA is a *system that persists*. Not the same kind of thing.

The separating test

Two questions decide which layer you're in:

1. **Does the work outlive the run, anchored to a durable identity?**
   - Workflow → **no.** Results live in script variables; the run ends; nothing is anchored to a spec.

- AIDA → **yes.** The spec is in git with a stable ID, a status, and trace comments in the code — queryable next week by a different agent on a different vendor.

2. **What is the repeatable thing?**
   - Workflow → **the orchestration script** (save a run, it becomes `/my-workflow`, replays the same fan-out).
   - AIDA → **the substrate + the lifecycle semantics** (the graph and "what Done means" persist; *how* a given drain fans out is incidental).

If your answer to "where does the result go?" is *"into a graph that other agents query later,"* you're in AIDA's layer. If it's *"into this report I'm about to read,"* you're in a Workflow.

What using a Workflow actually looks like

(Illustrative — the Workflow API and invocation surface move fast; treat these as the shape, not a frozen contract. Verify against the current Claude Code docs.)

Three ways to invoke one:

```
# 1. Dynamic, inline — just include the word "workflow" in a prompt:
"Sweep the codebase for unhandled Result types as a workflow and propose fixes."
#   → Claude writes an orchestration script on the fly, shows it to you, runs it.

# 2. Higher effort tier — auto-generates workflows for substantive tasks:
/effort ultracode
"Migrate every call site of the old config API to the new one."

# 3. Saved — after a good run, press `s` in /workflows to save it:
#   lands in .claude/workflows/<name>.* and becomes a slash command:
/deep-research  What changed in the Node permission model between v20 and v22?
```

`/deep-research` is the bundled example and the canonical shape: it **fans out** web searches across angles (parallel agents), **fetches + parses** each source (one agent per source), has agents **adversarially cross-check** each other's claims, **votes** and drops claims that didn't survive, and returns **one cited report** — not a transcript. That cross-check-before-reporting is the quality pattern a single agent can't do alone.

What the script underneath looks like (the runtime holds this loop; Claude's context only sees the final return):

```
export const meta = {
  name: 'review-diff',
  description: 'Review the current diff across dimensions, then adversarially verify eac
  phases: [{ title: 'Review' }, { title: 'Verify' }],
}

const DIMENSIONS = ['correctness', 'security', 'performance']

// Each dimension reviews; each of its findings is verified the moment that review lands
const results = await pipeline(
```

```
  DIMENSIONS,
  dim => agent(`Review the staged diff for ${dim} issues.`,
               { phase: 'Review', schema: FINDINGS_SCHEMA }),
  review => parallel(review.findings.map(f => () =>
    agent(`Adversarially verify this finding — try to REFUTE it: ${f.title}`,
          { phase: 'Verify', schema: VERDICT_SCHEMA })
      .then(v => ({ ...f, verdict: v })))),
)
const confirmed = results.flat().filter(f => f.verdict?.isReal)
return { confirmed }
```

The hooks are `agent()` (spawn one subagent; with a `schema` it returns validated structured output), `parallel()` (fan out, barrier-join), `pipeline()` (per-item stages, no barrier), `phase()`/`log()` (progress). Dozens-to-hundreds of agents per run (1000 cap, ~16 concurrent).

The AIDA composition, concretely. Because a Workflow's agents inherit the session's MCP connections, the *same* review workflow can ground itself in AIDA's graph — turning an ephemeral fan-out into a spec-aware one:

```
// Stage 0: ask AIDA (over MCP) what this diff is even supposed to satisfy,
// so the reviewers check against the acceptance criteria, not just generic smells.
const spec = await agent(
  `Call the AIDA MCP tool show_requirement for ${args.specId} and return its
   acceptance criteria + any blocked-by specs.`,
  { schema: SPEC_CONTEXT_SCHEMA })

// …then the per-dimension reviewers above each receive `spec.acceptance` in their prompt
```

That is the two-direction relationship made real: **Claude Code supplies the fan-out; AIDA supplies the ground truth the fan-out checks against.** AIDA never reimplements `pipeline()`/`parallel()` — it delegates to them and hands them the graph.

Overlap — said honestly

The intersection is real, not imagined:

- Both are **multi-agent orchestration**. Both fan out, both run in background, both support structured outputs.
- **The vocabulary collides.** AIDA's `phase`, `drain`, `pipeline-of-specs`; Workflows' `phase()`, `pipeline()`, `parallel()`. AIDA's `queue work --auto-complete` (implementer → CI → reviewer → merge) *is* a pipeline; AIDA's "spawn N adversarial reviewers and take majority" *is* a Workflow quality-pattern by another name.
- The honest read: **AIDA's orchestration *mechanism* and a Workflow are the same category of thing.** What differs is everything *around* the mechanism — the substrate, the identity, the persistence, the lifecycle.

They compose — two directions

This is where the relationship is **complementary**, and it runs both ways:

1. Workflow → AIDA (the workflow reads the graph). Subagents spawned inside a Workflow inherit the session's MCP connections — so they can call AIDA's MCP tools (`query_graph`, `show_requirement`, `search_requirements`). A research / review / migration workflow can ground itself in the spec graph: *"which specs touch `git_backend.rs`?"*, *"what are STORY-489's acceptance criteria?"* AIDA supplies durable context the ephemeral workflow doesn't have.

2. AIDA → Workflow (AIDA delegates a phase). AIDA can use a Workflow as the *implementation* of a phase that benefits from fan-out: - the **reviewer phase** → a workflow that spawns N adversarial reviewers and verifies findings before voting; - the **planning phase** → a judge-panel workflow drafting from several angles (this is conceptually what `aida ultraplan` already feeds `/ultraplan`); - the **drain itself** → SPIKE-32 (`aida show SPIKE-32`) compiles a spec's drain into a *saved* `workflow.js` artifact that Claude Code's runtime replays.

The dynamic-vs-saved lane (don't conflate)

Claude Code Workflows are fundamentally the **dynamic-generation lane**: you (or `/effort ultracode`) ask for a workflow, Claude writes the script, it decides the fan-out. You reach for them precisely when you *don't* need the identical plan every time.

Determinism is a *second* property, obtained by **saving** a successful run (`s` in `/workflows` → `.claude/workflows/` → a replayable slash command). The determinism comes from **replay of a fixed artifact**, not from the generation surface.

AIDA's "compile spec → `workflow.js`" thesis (SPIKE-32) lives in the **saved-script lane**: the compiler runs once per spec change, emits a `workflow.js` **build artifact** (think `Cargo.lock` — generated, checked in, replayed), and Claude Code's runtime replays *that*. The recurring AIDA drain wants *same-plan-every-time*, which is the saved lane, not the dynamic one.

> **Anti-pattern:** pitching AIDA as emitting *dynamic* workflows at orchestration time. It isn't. AIDA contributes the substrate + the emission step; Claude Code contributes deterministic replay.

The strategic read: commoditization is a gift here, not a threat

This is the tripwire worth saying plainly. Claude Code is building out the orchestration layer natively — subagents, Workflows, agent teams (SPIKE-29), `claude --bg`, `.claude/agents/`. AIDA's hand-rolled phase machine (sentinel files, exit-signal reaping, the punt handshake) increasingly overlaps with primitives Claude Code now ships for free.

The correct posture — already in motion via SPIKE-34 (re-shape `aida agent new claude` as a `claude --bg` wrapper) and the 2026-05-29 strategic-recompose — is **divest, not defend**:

- Let Claude Code own the orchestration **plumbing** (process supervision, fan-out, reaping, background execution).
- AIDA keeps the **substrate** (graph, stable IDs, enforced traces, git-canonical store), the **spec-lifecycle semantics** (what `Done` vs `Completed` means, auto-bump-on-merge), the **human-escalation ladder**, and **cross-vendor portability** — the things Workflows structurally do not have.
- Where AIDA orchestrates, prefer to express it *on top of* native Workflows rather than in competition with them.

AIDA's defensibility was never the orchestration mechanism — it's the persistent, vendor-neutral, git-canonical substrate. A Workflow can fan out 100 agents but can't tell you what exists, why it was chosen, or whether a function is still tied to a live requirement next quarter. It also can't orchestrate a *Codex* session — it's Claude-Code-only, while AIDA's queue now routes the same spec to a first-class Codex vendor (per-vendor routing, headless or interactive) against one shared store. So Workflows commoditizing orchestration is a **gift**: it lets AIDA delete hand-rolled plumbing and delegate. The only real danger is mistaking the orchestrator for the moat.

Worked example: quizdom under real load (2026-06-01)

The clearest proof of everything above came unprompted, from a sibling project. An agent in `~/ai/quizdom` ran the on-mission AIDA path — `aida queue work next 11 --no-human=both` — to drain 11 stories, 7 of them children of one epic. **AIDA's drain broke** (BUG-431: queue-work scoped each story's session to the parent *epic*, so same-epic siblings contended for one scope/branch/worktree; the resulting multi-spec PR jammed the headless reviewer; the failed phase never released the lease, cascade-blocking the rest). Only the first story per epic was attempted.

So the agent **fell back to the Claude Code Workflow tool** — and it worked: the 7 sequential, dependency-chained stories shipped fine, dependency-aware and worktree-isolated. That one episode demonstrates both halves of the thesis at once:

- **Orchestration is commoditized — concede it.** The harness Workflow did exactly the dependency-aware, isolated, sequential orchestration AIDA's drain does. AIDA gained nothing by hand-rolling that; it can't out-orchestrate the harness and shouldn't try.
- **Substrate is the moat — and the bypass proved it by its absence.** The Workflow shipped the *code* but populated *no substrate*: no `Draft→Completed` lifecycle flips, no leases, no trace comments, no graph updates. The agent itself flagged the loss. The work got done; the *knowledge of the work* leaked away. That gap **is** the product.
- **Reliability is what keeps work in the substrate.** Note *why* the agent defected — not preference, but that AIDA's drain was broken. When the substrate-native path fails, a rational agent routes around it to the commoditized one, and the substrate goes unpopulated. So the lesson isn't "out-orchestrate Workflows"; it's "be reliable enough that the substrate-native path is the path of least resistance." BUG-431 is

fixed (EPIC-33 invariant-1, 2026-06-01); once released, `aida queue work` is again the on-mission way to do exactly this — and the work stays in the graph.

The takeaway sharpens the strategic read above: **don't defend the orchestrator; defend the substrate — and keep the drain reliable enough that nobody has reason to leave it.**

Honest scope statement

What Workflows do that AIDA doesn't: within-task fan-out at dozens-to-hundreds of agents; adversarial cross-check panels; in-script token-budget scaling; codified-and-rerunnable orchestration of a *cognitive* task. If you need a 500-file migration or a cross-checked research report *right now*, that's a Workflow, not an AIDA drain.

What AIDA does that Workflows don't: a persistent requirement graph; stable IDs across renames and vendors; code→spec traces enforced at commit; cross-session lifecycle state; a human-escalation ladder; and a git-canonical substrate any vendor's agent can read. If you need to know *what exists and why* six months from now, that's AIDA, not a Workflow run that ended in March.

A serious multi-agent project uses **both**: AIDA for spec / lifecycle / traceability / cross-vendor coordination, Workflows for the within-spec tasks that genuinely need scale-out.

See also

- vs-claude-code-subagents.md — the sibling layer distinction (subagents are the *callable*; AIDA roles are the *position*).
- SPIKE-14 (`aida show SPIKE-14`) — Claude Code dynamic-workflows mechanism + AIDA composition path.
- SPIKE-32 (`aida show SPIKE-32`) — compile spec graph → `workflow.js` (the saved-script lane).
- SPIKE-29 (`aida show SPIKE-29`) — Claude Code agent teams, the third orchestration surface.
- SPIKE-34 (`aida show SPIKE-34`) — re-shape `aida agent new` as a `claude --bg` wrapper (the divest move).
- docs/competitive-analysis/2026-05-31-round2-moat-gaps-moves.md — the current moat / commoditization synthesis.